Mini Chess AI: Minimax with Alpha-Beta Pruning

AI Micro-Project Report | Batch 20
Navatheep Sing K N (VH15221) | Naveen P (VH15169) | Nithin Brijesh J (VH15209)

1. Abstract and Project Objectives

1.1 Abstract. This micro-project builds a small chess engine in Python using depth-limited minimax search with alpha-beta pruning. The engine generates moves for all six piece types, evaluates positions with material value plus a small centre-control bonus, and picks the move with the best minimax score. The base engine was extended, in a separate file and without changing the original code, with four features: a benchmark tool, capture-first move ordering, AI vs AI self-play and a human vs AI mode. Experiments show that alpha-beta pruning with move ordering removes up to 99.1% of the nodes that plain minimax searches at depth 4 (middlegame position), and that deeper search plays stronger: depth 3 scored 87.5% in a round-robin against depths 1 and 2, followed by depth 2 (37.5%) and depth 1 (25.0%).

1.2 Keywords. Minimax, alpha-beta pruning, game playing, move ordering, evaluation function, search depth.

1.3 Problem statement. Chess has far too many move sequences to examine completely, so an engine must search to a limited depth and still choose good moves. The problem is to build such an engine and to show, with measurements, how search depth and pruning change the amount of work and the quality of play.

1.4 Objectives.

- Implement legal-style move generation for all piece types on an 8x8 board.
- Implement an evaluation function using material and positional (centre) scores.
- Implement minimax with alpha-beta pruning and count nodes searched and branches pruned.
- Measure nodes and time for depths 1 to 4 with and without pruning and move ordering.
- Compare playing strength of depths 1, 2 and 3 using AI vs AI games and rank them.

1.5 Scope. *Included:* full 8x8 board, piece movement and captures, material and centre evaluation, minimax with alpha-beta, move ordering, benchmark, self-play, text-based human vs AI. *Excluded:* castling, en passant, pawn double-step, promotion, check and checkmate detection (a game ends when a king is captured), graphical interface and opening books.

2. Introduction and Proposed Solution

2.1 Background. Game playing is a classic area of AI. Chess is a two-player, turn-based, zero-sum game with perfect information, which makes it a standard test for adversarial search.

2.2 Existing approach and 2.3 limitations. The simplest approach is plain minimax, which explores every move to a fixed depth. Its limitations are: (1) the node count grows exponentially with depth, about 27 times per extra ply in our measurements; (2) without move ordering, alpha-beta pruning cuts far fewer branches than it could; (3) a material-only evaluation cannot see tactics beyond the search horizon.

2.4 Proposed solution. The engine chooses moves with minimax and alpha-beta pruning. The extension module adds capture-first move ordering so strong moves are searched early, and measurement tools that compare plain minimax, alpha-beta, and alpha-beta with ordering. **2.5 Intended users:** students learning game-playing AI, who can watch depth, pruning and evaluation working on a familiar game.

3. AI Concepts and Problem Formulation

3.1-3.2 AI concept. Game playing using adversarial search (minimax with alpha-beta pruning) and a heuristic evaluation function. It suits chess because the game is zero-sum: minimax assumes the opponent always replies with the move best for them, and alpha-beta skips branches that cannot change the final decision, making deeper search practical.

3.3 Problem formulation. *Initial state:* the standard starting position. *Actions:* all pseudo-legal moves of the side to move. *Transition:* make_move copies the board and moves the piece. *Terminal test:* depth limit reached, no moves, or a king captured. *Utility:* evaluation score (positive favours White). *Constraints:* moves stay on the board and cannot capture own pieces.

3.4 PEAS description.

Component	Project-specific description
Performance measure	Evaluation of the chosen move, nodes searched, branches pruned, time, self-play result
Environment	8x8 chess board, two players, fully observable, deterministic, turn-based
Actuators	The chosen move in chess notation (e.g. e4d5), applied to the board
Sensors / inputs	Current board state, side to move, search depth, human moves typed as e2e3

3.5 Input, output, assumptions. Input: board, side to move, depth (1-3 in play, up to 4 in the benchmark). Output: best move, score, nodes, pruned count, tables and game results. White is the maximising player; the simplified rules in Section 1.5 apply.

4. System Architecture and Design

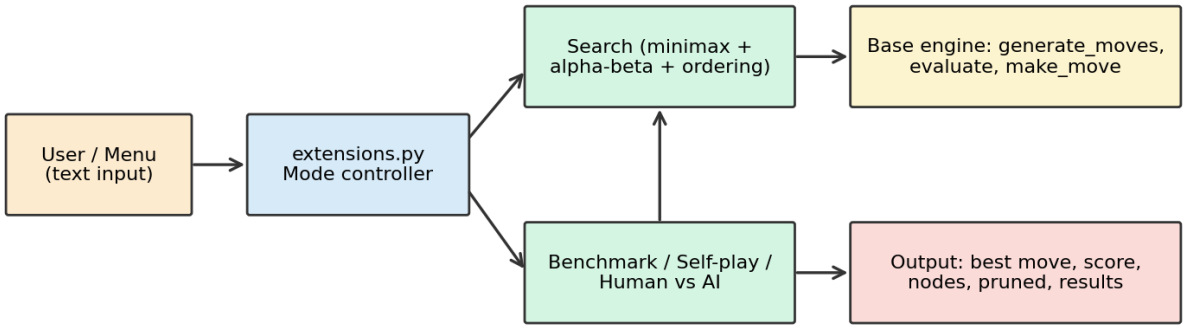
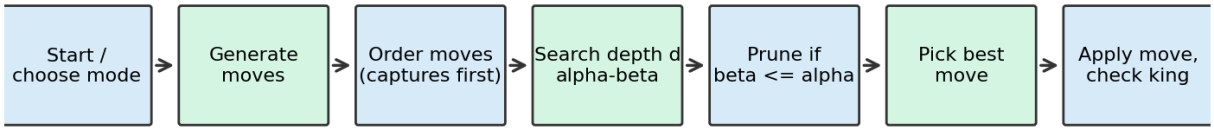


Figure 1: Architecture. The user chooses a mode in extensions.py, which uses the search function; the search reuses generate_moves, evaluate and make_move from the unchanged base engine, and results are printed.



Loop until a king is captured, the move limit is reached or the player quits

Figure 2: Workflow of one AI move: generate moves, order them, search to depth d with alpha-beta, pick the best move, apply it and check for game end.

4.3 Modules and responsibilities.

Module	Responsibility / function
Base engine (src/)	Board display, move generation, evaluation, make_move, minimax with alpha-beta, notation (unchanged)
extensions.py	Move ordering, search with pruning/ordering switches, benchmark, self-play, human vs AI, menu
tests/test_extensions.py	Six automated checks: move parsing, win detection, move ordering, score equality, pruning saves nodes

4.4 Tools. Python 3 (standard library only for the program), Visual Studio Code, Git/GitHub, pytest for tests.

5. Algorithm and Methodology

5.1 Methodology. (1) Represent the board as an 8x8 list of characters (uppercase White, lowercase Black). (2) Write move generation per piece. (3) Write the evaluation function. (4) Implement minimax, then add alpha-beta with node and prune counters. (5) Add move ordering, benchmark, self-play and human vs AI in a separate file. (6) Test and run experiments.

5.2 Pseudocode.

```
search(board, depth, alpha, beta, maximizing):
    count node
    if depth == 0 or a king is captured: return evaluate(board)
    moves = generate_moves(board, side to move)
    if no moves: return evaluate(board)
    order moves: captures first (valuable victim, cheap attacker)
    for each move:
        score = search(make_move(board, move), depth-1, alpha, beta, not maximizing)
        update best score and alpha (if maximizing) or beta (if minimizing)
        if beta <= alpha: count a prune; stop searching this node
    return best score, best move
```

5.3 Technique. White maximises and Black minimises the score. Alpha and beta store the best scores already guaranteed to each side; when a branch cannot beat them the remaining moves are skipped, giving the same result as plain minimax with fewer nodes. Move ordering tries likely-good moves first so the window tightens early. **5.4 Data structures.** Board: list of 8 lists of characters; move: tuple (row1, col1, row2, col2); piece values: dictionary; statistics: dictionary of counters.

5.5 Complexity. With branching factor b and depth d, plain minimax visits $O(b^d)$ nodes. Alpha-beta with perfect ordering approaches $O(b^{d/2})$; real results lie in between. Space is O(d) for the recursion stack plus board copies.

6. Implementation and GitHub Repository

6.1 Environment. VS Code, Python 3, pytest. **6.2 Steps.** Base engine first; then extensions.py imports it and adds order_moves, search/ai_move, benchmark, self_play, human_vs_ai and a menu. **6.3 Repository URL:** https://github.com/Jagan-732/[REPOSITORY-NAME] **6.4 Access:** [public / private with faculty access]

6.5 Repository structure.

File / folder	Purpose	Included?
src/ (engine, extensions.py)	Main implementation	Yes
README.md	Overview and run steps	Yes
requirements.txt	Dependencies (pytest)	Yes
tests/	Test scripts	Yes
docs/	Report	Yes
screenshots/	Execution evidence	[Yes / No]
.gitignore	Excludes local files	Yes

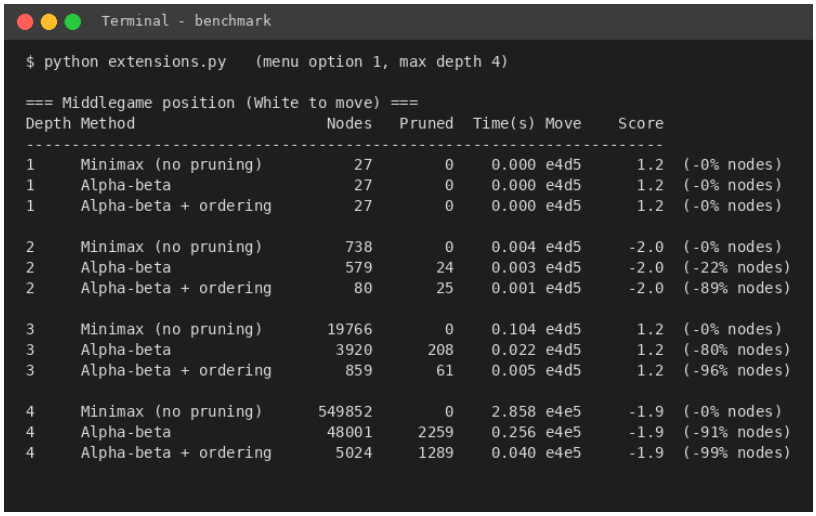
6.6 Run steps. `git clone <repo-url>; cd src; python extensions.py` (menu 1 = benchmark, 2 = self-play, 3 = human vs AI); tests: `pip install pytest; pytest` from the project root. **6.7 Contributions:** [describe who did engine, extensions, testing and report].

7. Results and Observations

7.1 Results summary. All numbers below were measured by running the code. The six tests passed. In every position tested, minimax, alpha-beta and alpha-beta with ordering returned the same score and best move at each depth, confirming that pruning does not change the decision. Times are small on this machine, so node counts are the main measure.

Table 1: Middlegame position (White to move), nodes searched and percentage saved versus minimax.

Depth	Minimax nodes	Alpha-beta nodes (saved)	Alpha-beta + ordering (saved)	Time minimax / AB+ord
1	27	27 (0%)	27 (0%)	0.000 s / 0.000 s
2	738	579 (21.5%)	80 (89.2%)	0.004 s / 0.001 s
3	19,766	3,920 (80.2%)	859 (95.7%)	0.104 s / 0.005 s
4	549,852	48,001 (91.3%)	5,024 (99.1%)	2.858 s / 0.040 s



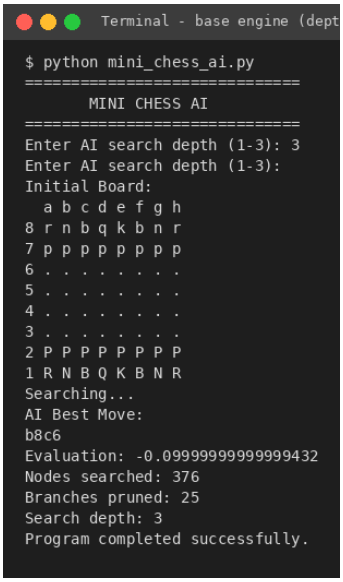


Figure 3 (left): Benchmark output (menu option 1, depth 4). Figure 4 (right): base engine run at depth 3 from the initial position: best move b8c6, 376 nodes searched, 25 branches pruned.

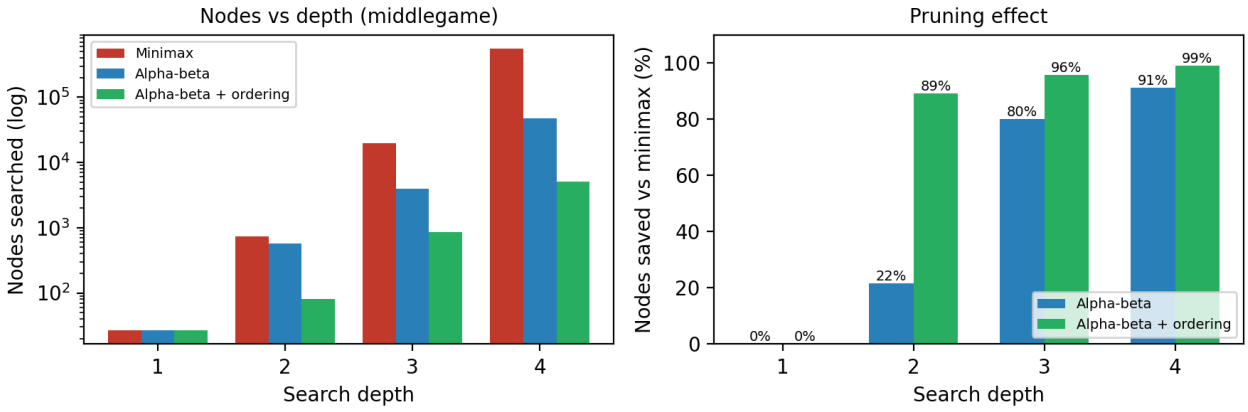


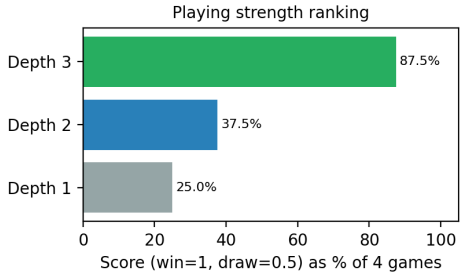
Figure 5: Nodes grow roughly 27x per extra ply for plain minimax. Pruning and ordering flatten the growth; the saving rises with depth.

Table 2: Round-robin self-play, 40 moves per side, each pairing played once with each colour.

White \ Black	Depth 1	Depth 2	Depth 3
Depth 1	Black wins (king captured)	Black wins (king captured)	Black wins (king captured)
Depth 2	Roughly equal (eval +0.1)	Roughly equal (eval +0.3)	Black wins (king captured)
Depth 3	Roughly equal (eval 0.0)	White ahead (eval +19.5)	Roughly equal (eval +0.1)

Table 3: Ranking by score (win = 1, draw = 0.5, loss = 0; mirror games depth-vs-same-depth excluded, so each depth plays 4 games).

Rank	Search depth	Wins	Draws	Losses	Score
1	Depth 3	3	1	0	3.5 / 4 = 87.5%
2	Depth 2	1	1	2	1.5 / 4 = 37.5%
3	Depth 1	0	2	2	1.0 / 4 = 25.0%



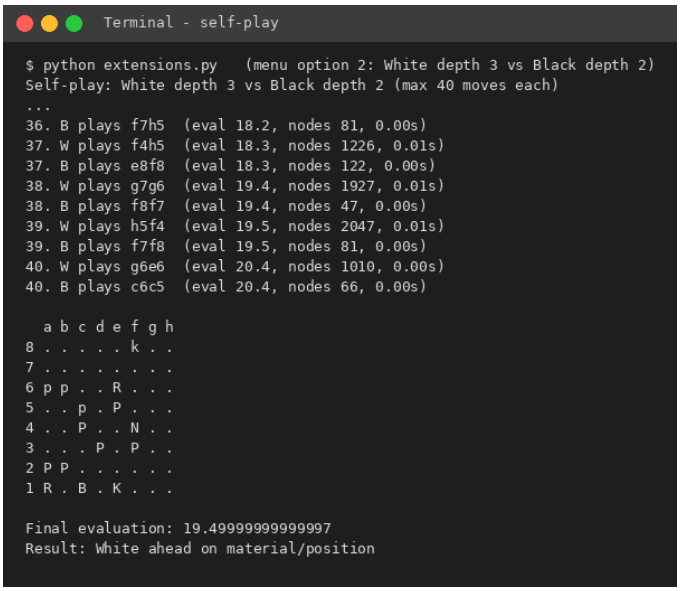


Figure 6: Self-play output (menu option 2): White depth 3 vs Black depth 2. White finishes about 19.5 points ahead in material and position.

7.2 Observations. Objectives 1-5 were met. (a) Plain minimax grew about 27 times per ply. (b) Alpha-beta saved 21.5% of nodes at depth 2 and 91.3% at depth 4; adding move ordering raised this to 89.2% and 99.1%, with a 71x speed-up in time at depth 4 (2.858 s to 0.040 s). (c) Ordering helps only where captures exist: in the initial position it saved nothing extra (depth 3: 271 nodes both ways, 88.1% saved versus minimax's 2,281). (d) Deeper search played stronger: depth 3 ranked first and depth 1 last. Depth 1 lost its king in all three games as White. **7.3 Reliability.** The engine is deterministic and the sample is small (4 games per depth), so the ranking shows a clear trend, not a statistically precise strength measure. Several draws came from repeated moves (for example a queen shuffling between two squares), because the engine has no repetition rule.

8. Discussion, Limitations and Future Enhancements

8.1 Discussion. Minimax gave the engine look-ahead against a best-reply opponent; alpha-beta let it search the same tree with far fewer nodes, and ordering improved the pruning further, which is what makes depth 3 and 4 practical in plain Python. **8.2 Limitations.** (1) Simplified rules: no castling, en passant, double-step pawns, promotion or check detection. (2) The evaluation uses only material and a centre bonus, and has no repetition handling. (3) No quiescence search, so captures at the search horizon can be misjudged. **8.3 Future work.** Full legal-move rules with check and checkmate; iterative deepening with a transposition table; piece-square tables and king safety; a graphical interface. **8.4 Applications.** Teaching tools for AI courses, simple game bots, and a base for studying search algorithms. **8.5 Learning outcomes.** We learned how adversarial search works, why pruning and move ordering matter, how to measure algorithm performance, how to extend code without modifying it, and how to use Git, GitHub and automated tests.

9. Conclusion and References

9.1 Conclusion. The project delivers a working mini chess engine with minimax and alpha-beta pruning, extended with move ordering, a benchmark, self-play and human vs AI modes. Measurements show that pruning with ordering removes up to 99.1% of the nodes at depth 4 without changing the chosen move, and self-play ranks deeper search as stronger (depth 3 first, depth 2 second, depth 1 third), within the limits of the simplified rules.

References. [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson. [2] Course notes on adversarial search and game playing.